

# Plan de integración API — World Office Cloud

EM-Pedidos · E.M. Compañía S.A.S

Documento de integración (sección 19 del Build Spec) — el núcleo de la evaluación.

**Tesis.** La plataforma ya está lista para producción. El 90% (creación de pedidos, armado del payload, mapeo campo a campo, manejo de errores, idempotencia) está construido, probado y corriendo contra datos reales en `WO_MODE=mock`. El 10% restante —la conexión viva— se enciende cambiando **un flag** (`WO_MODE=live`) y poblando los IDs reales del tenant de E.M., sin tocar el resto de la aplicación.

## 1. Resumen ejecutivo

| Aspecto                        | Cómo lo resolvimos                                                                                                    |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Único punto de contacto con WO | <code>WorldOfficeAdapter</code> (anti-corruption layer). El resto de la app no conoce el formato de WO.               |
| Doble modo                     | <code>WO_MODE = mock   live</code> . <b>Interfaz idéntica</b> . El mock valida el mismo payload que el live.          |
| Determinismo                   | El payload se arma con código puro ( <code>mapping.ts</code> ), sin IA en el camino crítico.                          |
| Idempotencia                   | Consecutivo controlado por nosotros + <code>idempotency_key</code> . Un reintento nunca duplica.                      |
| Errores                        | 12 errores de WO documentados y tipados; cada uno blindo un campo del payload; todo queda en <code>sync_logs</code> . |
| Auditabilidad                  | Cada pedido guarda <code>wo_payload</code> (enviado) y <code>wo_response</code> (recibido) en Supabase.               |
| Orquestación                   | <code>n8n</code> (flujo <code>crearPedido</code> ) con reintentos y backoff, llamando al adapter vía HTTP.            |

## 2. Estado de la API de World Office (investigado) — 19.1

- **Tipo:** API REST (JSON), métodos GET/POST/PUT/DELETE. Gratuita en plan **Enterprise**.
- **Autenticación:** JWT con **vigencia 12 h**, emitido por `gestionarTokenAPILicencia` (o token de la app). Se envía en cada request con el header `Authorization: WO <token>` (prefijo literal `WO`).
- **Módulos:** Terceros, Ventas (incluye **Pedidos** y Facturas), Inventarios, Compras, Contabilidad, Cartera, Cuentas por Pagar.
- **Creación de documentos de venta:** exige `documentoTipo` (obtenido de "Listar tipos de documentos"), `idEmpresa`, tercero, prefijo, forma de pago, moneda, bodega, centro de costo y renglones con `idInventario`.
-

**Control de duplicados:** WO marca duplicado cuando coinciden `prefijo + idEmpresa + documentoTipo + numero`.

- **Rate limit:** generoso (la documentación menciona ~500 req/seg); aun así encolamos en n8n.
- **Base URL:** `por-tenant` (la documentación muestra `localhost:8080` como placeholder). Es una variable de entorno a confirmar con WO/E.M. — ver `docs/PREGUNTAS-CLIENTE.md` (P1).

### 3. Por qué corremos en `mock` durante el concurso — 19.2

World Office **no ofrece un ambiente de pruebas público**: la API solo opera contra la cuenta real del cliente. Por eso *demostrar dominio* de la API (payload + mapeo + errores + plan de cableado) vale más que una conexión en vivo que nadie puede ejecutar sin el contrato.

`WO_MODE=mock` corre el `WorldOfficeMockAdapter`, que:

- **valida el mismo payload** que el live (idéntico esquema y mismas reglas),
- **simula** respuestas de éxito (devuelve un `numero`) y de error (`TERCERO_ERRADO`, `INVENTARIO_NO_ENCONTRADO`, ...),
- **detecta duplicados** por `idempotency_key` (demuestra la idempotencia),
- **persiste** `wo_payload` y `wo_response` igual que el live.

### 4. Arquitectura de la integración

```
Vendedor confirma pedido (Next.js, determinista)
  crea pedido (estado=confirmado) + wo_payload congelado
1/4
SUPABASE (fuente de verdad: pedidos, pedido_items, sync_logs)
  dispara camino crítico
1/4
n8n · crearPedido (sin IA) POST {payload} °/api/worldoffice/crear-pedido
  IF ok 'estado=sincronizado_wo + Gmail (HTTP wrapper)
  IF err 'estado=pendiente_sync + sync_logs + retry 1/4
                                getAdapter() ÅWO_MODE
                                mock | live
1/4
Gmail (Composio) 1/4
WORLD OFFICE CLOUD API
[CONCURSO: mock] · [PROD: flaglive]
```

**Anti-corruption layer.** Todo lo que sabe de WO vive en `apps/web/lib/worldoffice/`:

| Archivo | Responsabilidad |
|---------|-----------------|
|         |                 |

|            |                                                                                |
|------------|--------------------------------------------------------------------------------|
| types.ts   | WOPedidoPayload, WOREnglon, WOREsult, catálogos de reconciliación.             |
| errors.ts  | 12 errores documentados de WO + qué campo blinda cada uno + validación tipada. |
| mapping.ts | Mapeo determinista Supabase ' payload + validación + idempotencyKey.           |
| mock.ts    | WorldOfficeMockAdapter (valida + simula éxito/error/duplicado).                |
| live.ts    | WorldOfficeLiveAdapter (endpoints reales del go-live).                         |
| adapter.ts | Interfaz + getAdapter() que decide mock   live por WO_MODE.                    |

El getAdapter() se expone a n8n vía apps/web/app/api/worldoffice/crear-pedido/route.ts, de modo que **n8n nunca rearma el payload ni reimplementa WO** (cero divergencia).

## 5. Autenticación y ciclo de token

- Emisión:** POST {WO\_BASE\_URL}/gestionarTokenAPILicencia con body text/plain = correo registrado (WO\_CORREO\_REGISTRADO). Respuesta: JWT (12 h).
- Uso:** header Authorization: WO <token> en cada request.
- Renovación:** flujo n8n refreshToken (cron cada ~11 h) lo regenera antes de expirar y lo guarda cifrado (secret de n8n). **Solo en modo live.**
- Seguridad:** el token nunca está en el front; vive como secret de n8n / variable de entorno del servidor.

## 6. Mapeo Supabase 'World Office (campo a campo)

Implementado en mapping.ts (buildWOPayload). Cada campo tiene un **error de WO que se previene** validándolo (validateWOPayload):

| Interno (Supabase)            | Campo WO         | Error que previene                     |
|-------------------------------|------------------|----------------------------------------|
| empresa.wo_id_empresa         | idEmpresa        | EMPRESA_ERRADA                         |
| empresa.documento_tipo_pedido | documentoTipo    | TIPO_DOCUMENTO_NO_ADMITO_API           |
| pedidos.prefijo + consecutivo | prefijo + numero | PREFIJO_FACTURA_ERRADO / DUPLICATE_KEY |
| clientes.wo_id_tercero        | idTerceroExterno | TERCERO_ERRADO                         |

|                                       |                     |                                   |
|---------------------------------------|---------------------|-----------------------------------|
| clientes.wo_id_direccion              | idDireccionTercero  | DIRRECCION_TERCERO_EXTERNO_ERRADO |
| empresa.forma_pago_default            | formaPago           | FORMA_PAGO_NO_SOPORTADA           |
| empresa.moneda (COP)                  | idMoneda            | ERROR_MONEDA                      |
| productos.wo_id_inventario (snapshot) | idInventario        | INVENTARIO_NO_ENCONTRADO          |
| productos.wo_id_unidad                | unidadMedida        | ERROR_UNIDAD_INVENTARIO           |
| empresa.bodega_default                | idBodega            | BODEGA_NO_EXISTE                  |
| empresa.centro_costo_default          | idCentroCosto       | CENTRO_COSTO_NO_EXISTE            |
| clientes.descuento_pct                | porcentajeDescuento | (regla de negocio)                |

**Snapshots inmutables.** Cada línea de cotización/pedido guarda `codigo_contable_snapshot` y `wo_id_inventario_snapshot`. Aunque el vendedor haya buscado por descripción y aunque el catálogo cambie después, **el código contable siempre viaja** y el pedido es reproducible bit a bit.

**IDs simulados en mock.** Mientras `wo_id_*` esté en `null` (antes del go-live), el mapeo sustituye cada ID por un valor **simulado y namespaced** (`SIM-INV-<codigo_contable>`, `SIM-EMPRESA`, ...) para demostrar un payload completo. En `live`, un ID nulo es un **error de validación** (no se envía un pedido inválido).

### Ejemplo de payload (mock, pedido PED-1)

```
{
  "documentoTipo": "SIM-PEDIDO",
  "idEmpresa": "SIM-EMPRESA",
  "prefijo": "PED",
  "numero": "1",
  "fecha": "2026-06-24",
  "idTerceroExterno": "SIM-TERCERO",
  "idDireccionTercero": "SIM-DIR",
  "formaPago": "contado",
  "idMoneda": "COP",
  " renglones": [
    { "idInventario": "SIM-INV-0100012", "unidadMedida": "SIM-UND-0100012", "cantidad": 2,
      "valorUnitario": 23000, "porcentajeDescuento": 12.5, "idBodega": "SIM-BODEGA",
      "idCentroCosto": "SIM-CC", "idImpuesto": "SIM-IMP-0" },
    { "idInventario": "SIM-INV-0200004", "unidadMedida": "SIM-UND-0200004", "cantidad": 3,
      "valorUnitario": 43000, "porcentajeDescuento": 12.5, "idBodega": "SIM-BODEGA",
      "idCentroCosto": "SIM-CC", "idImpuesto": "SIM-IMP-1" }
  ]
}
```

En go-live, los `SIM-*` se reemplazan por los IDs reales del tenant (sección 11). La estructura no cambia.

## 7. Idempotencia y consecutivos

- El **consecutivo lo controlamos nosotros**, no WO: la función `siguiente_consecutivo()` (Postgres, `security_definer`) reserva el próximo número de forma **atómica** (lock de fila sobre empresa).
- `pedidos` tiene **dos garantías de unicidad**: `unique(prefijo, consecutivo)` y `unique(idempotency_key)`.
- La `idempotency_key` reproduce exactamente la regla de duplicados de WO:  
`prefijo :: idEmpresa :: documentoTipo :: numero (idempotencyKey())` en `mapping.ts`.
- Un **reintento reusa el mismo número y la misma idempotency\_key** ' WO lo reconoce como el mismo documento y no se duplica. El mock demuestra esto devolviendo `DUPLICATE_KEY` ante una segunda llamada.

**Resultado:** un pedido nunca se duplica ni se pierde, aunque la red falle a mitad de camino.

---

## 8. Manejo de errores y robustez — 19.4

`crearPedido` (mock y live) valida el payload antes de "enviar" y, ante un error de WO, devuelve un `WOResult { ok:false, errorCode, moreInfo }`. El camino crítico:

1. Marca el pedido `pendiente_sync`.
2. Inserta un registro en `sync_logs` (`request`, `response`, `status`, `error_code`, `error_more_info`).
3. **Reintenta** con backoff (n8n: `retryOnFail`, `maxTries`, `waitBetweenTries`), **reusando la misma idempotency\_key**.

Errores documentados (en `errors.ts`, cada uno con su mensaje):

`EMPRESA_ERRADA`, `TIPO_DOCUMENTO_NO_ADMITO_API`, `PREFIJO_FACTURA_ERRADO`, `DUPLICATE_KEY`, `TERCERO_ERRADO`, `DIRECCION_TERCERO_EXTERNO_ERRADO`, `FORMA_PAGO_NO_SOPORTADA`, `ERROR_MONEDA`, `INVENTARIO_NO_ENCONTRADO`, `ERROR_UNIDAD_INVENTARIO`, `BODEGA_NO_EXISTE`, `CENTRO_COSTO_NO_EXISTE`.

La migración del catálogo de escritorio a WO Cloud la realiza el equipo de World Office; no es alcance de esta plataforma.

---

## 9. El camino crítico (app + n8n)

Flujo `n8n/workflows/crearPedido.json` (validado, sin IA):

1. **Webhook** • la app envía `{pedido_id}` al confirmar (si `N8N_WEBHOOK_CREAR_PEDIDO` está configurado; si no, la app sincroniza in-app con el mismo resultado). Alternativa: Database Webhook de Supabase.
2. **Leer pedido** (Supabase REST) ' trae el `wo_payload` ya congelado.
3. **Crear en WO** ' `POST /api/worldoffice/crear-pedido` (el wrapper del adapter; mock o live según `WO_MODE`).

4. **IF éxito** ' `marcasincronizado_wo` (con `numero_wo`, `synced_at`, `wo_response`) + notifica a contabilidad (Gmail vía Composio). **Error** ' `pendiente_sync` + `sync_logs` + reintento.

El mismo flujo sirve para concurso (mock) y producción (live): n8n no cambia, solo el `WO_MODE` de la app.

---

## 10. Archivos y estructuras que alimentan WO (criterio #3)

Por cada pedido se produce, reusando **el mismo mapeo** del adapter (cero divergencia entre lo que se muestra y lo que se enviaría):

- **Payload WO (JSON)**: el `wOPedidoPayload` exacto, visible y **descargable** desde el panel contable.
- **Estructura de carga (CSV)**: los renglones listos para WO, **descargable** por contable/admin.
- (Pendiente 2.5) **PDF** de cotización y pedido con marca E.M.

---

## 11. Pasos de cableado en producción — 19.3

1. E.M. genera el **token API** desde su cuenta Enterprise (Configuración ' Configuración General ' API).
2. Confirmar `WO_BASE_URL` real del tenant y registrar `WO_CORREO_REGISTRADO`.
3. **Reconciliación de IDs** (el paso clave): llamar `listarInventarios`, `listarTerceros`, `listarTiposDocumento` y mapear catálogo/clientes reales ' poblar `productos.wo_id_inventario / wo_id_unidad / wo_id_impuesto`, `clientes.wo_id_tercero / wo_id_direccion`, `empresa.wo_id_empresa / documento_tipo_pedido / bodega_default / centro_costo_default`.  
Nuestra muestra usa IDs simulados; en go-live se sustituyen por los reales.
4. Cambiar `WO_MODE=live` y activar el flujo `refreshToken` en n8n.
5. **Prueba controlada**: crear 1 pedido real ' verificar que aparece en WO listo para factura.
6. Activar el flujo completo para los 3 vendedores.

---

## 12. Riesgos y supuestos abiertos — 19.5

Detallados en `docs/PREGUNTAS-CLIENTE.md`. Resumen:

- **URL base real** del tenant y **nombre exacto** del `documentoTipo` "Pedido".
  - Si el **descuento por cliente** se aplica como % por renglón o por documento (hoy: por renglón, marcado).
  - Si la **consulta de inventario** en vivo es por producto o por lote (impacta el panel de stock).
-

---

### 13. Qué es real / qué es mock

| Real (construido y probado)                                                               | Mock (se enciende con un flag)                         |
|-------------------------------------------------------------------------------------------|--------------------------------------------------------|
| Modelo de datos, RLS por rol, búsqueda dual                                               | La conexión <b>viva</b> a World Office                 |
| Armado determinista del payload + mapeo campo a campo                                     | —                                                      |
| Validación server-side + 12 errores tipados                                               | El mock simula éxito/error con el <b>mismo</b> esquema |
| Idempotencia (consecutivo atómico + <code>idempotency_key</code> )                        | —                                                      |
| Auditoría ( <code>wo_payload</code> , <code>wo_response</code> , <code>sync_logs</code> ) | —                                                      |
| Flujos n8n (camino crítico + retry)                                                       | —                                                      |
| Notificación Gmail (Composio) — <b>envío real verificado</b>                              | —                                                      |

Cambiar de mock a live es: `WO_MODE=live` + IDs reconciliados + `refreshToken` activo. **Nada más.**

---

### 14. Checklist de go-live

- `WO_BASE_URL` y `WO_CORREO_REGISTRADO` configurados.
- Token API generado; `refreshToken` (n8n) activo.
- `productos.wo_id_*` reconciliados (`listarInventarios`).
- `clientes.wo_id_tercero` / `wo_id_direccion` reconciliados (`listarTerceros`).
- `empresa.wo_id_empresa` / `documento_tipo_pedido` / `bodega_default` / `centro_costo_default` poblados.
- `WO_MODE=live`.
- Pedido de prueba creado y verificado en WO.
- Notificación apuntando al correo del área contable de E.M.